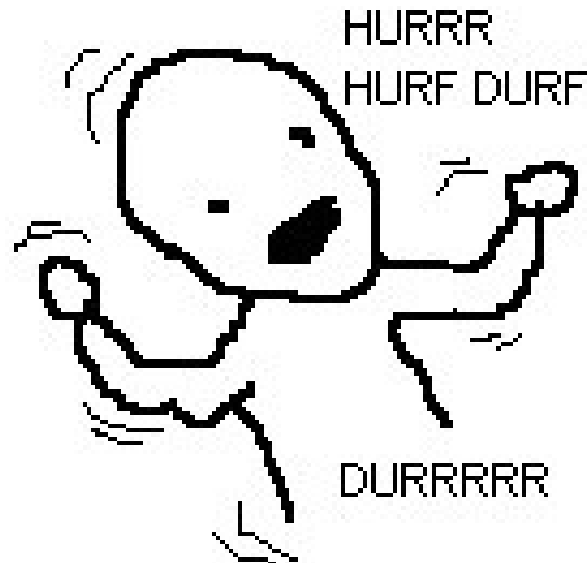


Reverse Engineering Malware For Newbies



```
push    20h           ; OpenOptions
push    3             ; ShareAccess
lea     eax, [ebp+IoStatusBlock]
push    eax           ; IoStatusBlock
lea     eax, [ebp+ObjectAttributes]
push    eax           ; ObjectAttributes
push    0C0000000h    ; DesiredAccess
lea     eax, [ebp+Handle]
push    eax           ; FileHandle
call    ds:ZwOpenFile
mov     ebx, eax
cmp     ebx, esi
jnz     loc_10BAF
```

A guide for those of you who want to
break into the fun world of malware.

What We're Going To Cover

- Basic x86/64 ASM
- Tools of the trade
- Setting up an environment
- Intro to the Debugger
- Basic Debugger Usage
- Reporting
- Basic Malware Analysis
- Unpacking 101
- Dynamic Analysis
- Memory Analysis
- Where to get malware
- Help! I'm Stuck!
- Additional Resources

Basic X86 / 64 ASM

My good friend Eric Davisson will now speak about the basics of X86 assembly.



Tools of The Trade

Every reverse engineer should have these in their bag:

- Debugger - ollydbg, windbg, syser, IDA, immunity.
- Disassembler – IDA, olly, immunity, windbg
- System Monitoring tools - procmon, Process hacker
- Misc tools - exeinfo, HookAnalyser, Cuckoo Sanbox, gmer, rootrepeal, wireshark, volatility, HXD, ilspy, jd-gui, p32dasm, cff explorer, winprefetch viewer, autoruns.

Lot's of tools, but they each serve a purpose. Over time you'll become accustomed to each one and wonder what you ever did without them.

Setting Up Your Environment

- Every Environment should be off your network unless absolutely necessary.
- Use a VM!!!
- Sandboxie is no substitute
- Cuckoo works wonders
- Dual monitors are a godsend

Enter The Debugger

The debugger is your best friend. Treat him with respect. There are 2 kinds of debuggers - kernel debuggers and usermode debuggers.

- A kernel debugger operates at ring 0 - essentially the driver level and has direct access to the kernel.
- A usermode debugger is the opposite, having only access to the usermode space of the operating system. Most of the time, this is enough, but not always. In the case of rootkits, or even super advanced protection schemes, its preferable to step into a kernel mode debugger instead as user mode would then become untrustworthy.

Immunity Break Down

The screenshot displays the Immunity Debugger interface with the following components:

- Top Bar:** Shows the current file being debugged: `avtst2.exe - [CPU - main thread, module avtst2]`. The menu bar includes File, View, Debug, Plugins, ImmLib, Options, Window, and Help. The toolbar contains various debugging icons.
- Left Panel:** Contains the CPU window, which shows the current instruction being executed. The instruction is `PUSH EBP` at address `00401050`. The CPU window also displays the current instruction pointer (EIP) and the current instruction's operands.
- Right Panel:** Contains the Registers window, which shows the current state of the CPU registers. The registers are listed with their names and values. The registers window also displays the current instruction pointer (EIP) and the current instruction's operands.
- Bottom Left Panel:** Contains the Dump window, which shows the memory dump of the current instruction. The dump is displayed in hexadecimal and ASCII format.
- Bottom Right Panel:** Contains the Stack window, which shows the current state of the stack. The stack is displayed in hexadecimal and ASCII format.

The CPU window shows the following instructions:

```
00401050 $ 55 PUSH EBP
00401051 . 89E5 MOV EBP,ESP
00401053 . 6A FF PUSH -1
00401055 . 68 0C304000 PUSH avtst2.00403000
0040105A . 68 58114000 PUSH avtst2.00401158
0040105F . 64:FF35 000000 PUSH DWORD PTR FS:[0]
00401066 . 64:8925 000000 MOV DWORD PTR FS:[0],ESP
0040106D . 83EC 0C SUB ESP,0C
00401070 . 53 PUSH EBX
00401071 . 56 PUSH ESI
00401072 . 57 PUSH EDI
00401073 . 8965 E8 MOV DWORD PTR SS:[EBP-10],ESP
00401076 . 68 00000002 PUSH 20000000
0040107B . E8 900C0000 CALL avtst2.00401010
00401080 . 59 POP ECX
00401081 . A3 38434000 MOV DWORD PTR DS:[404338],EAX
00401086 . E8 E5020000 CALL avtst2.00401370
0040108B . 85C0 TEST EAX,EAX
0040108D . 75 0D JNZ SHORT avtst2.0040109C
0040108F . 6A 01 PUSH 1
00401091 . E8 1A050000 CALL avtst2.00401580
00401096 . 59 POP ECX
00401097 . E9 9B000000 JMP avtst2.00401137
0040109C > C745 FC 000000 MOV DWORD PTR SS:[EBP-4],0
004010A3 . E8 18050000 CALL avtst2.004015C0
004010A8 . E8 D3050000 CALL avtst2.00401680
004010AD . E8 1E060000 CALL avtst2.004016D0
004010B2 . E8 290A0000 CALL avtst2.00401AE0
004010B7 . E8 C40A0000 CALL avtst2.00401880
004010BC . BB 90304000 MOV EBX,avtst2.00403090
004010C1 . 81FB 90304000 CMP EBX,avtst2.00403090
004010C7 . 72 00 JNB SHORT avtst2.004010D6
EBP=0018FF94
```

The Registers window shows the following registers:

```
EAX 771A3378 kernel32.BaseThreadInitThunk
ECX 00000000
EDX 00401050 avtst2.<ModuleEntryPoint>
EBX 7EFDE000
ESP 0018FF8C
EBP 0018FF94
ESI 00000000
EDI 00000000
EIP 00401050 avtst2.<ModuleEntryPoint>
C 0 ES 002B 32bit 0(FFFFFFFF)
P 1 CS 0023 32bit 0(FFFFFFFF)
A 0 SS 002B 32bit 0(FFFFFFFF)
Z 1 DS 002B 32bit 0(FFFFFFFF)
S 0 FS 0053 32bit 7EFDD000(FFF)
T 0 GS 002B 32bit 0(FFFFFFFF)
D 0
O 0 LastErrr ERROR_SXS_KEY_NOT_FOUND (000036B7)
EFL 00000246 (NO,NB,E,BE,NS,PE,GE,LE)
ST0 empty g
ST1 empty g
ST2 empty g
ST3 empty g
ST4 empty g
ST5 empty g
ST6 empty g
ST7 empty g
3 2 1 0 E S P U 0 2 D I
FST 0000 Cond 0 0 0 0 Err 0 0 0 0 0 0 0 0 (GT)
FCW 027F Prec NEAR,53 Mask 1 1 1 1 1
```

The Dump window shows the following memory dump:

```
Address Hex dump ASCII
00404000 3C 40 00 00 00 00 00 00 <.....
00404008 00 00 00 00 40 41 00 00 ....@A..
00404010 B0 40 00 00 08 40 00 00 "0..."
00404018 00 00 00 00 00 00 00 00 .....
00404020 5C 41 00 00 1C 41 00 00 \A..A..
00404028 00 00 00 00 00 00 00 00 .....
00404030 00 00 00 00 00 00 00 00 .....
00404038 00 00 00 00 24 41 00 00 ...$.A..
00404040 32 41 00 00 68 41 00 00 2A...hA..
00404048 74 41 00 00 82 41 00 00 tA...MA..
00404050 90 41 00 00 9C 41 00 00 A...MA..
00404058 AA 41 00 00 B6 41 00 00 A...MA..
00404060 C2 41 00 00 D2 41 00 00 A...MA..
00404068 E0 41 00 00 FA 41 00 00 A...MA..
00404070 0C 42 00 00 1A 42 00 00 .B...B..
00404078 2A 42 00 00 3E 42 00 00 *B...>B..
00404080 50 42 00 00 62 42 00 00 PB...bB..
00404088 74 42 00 00 8A 42 00 00 tB...MB..
00404090 A2 42 00 00 BC 42 00 00 CB...%B..
00404098 D8 42 00 00 F0 42 00 00 DB...sB..
004040A0 00 43 00 00 00 00 00 00 .C.....
004040A8 4E 41 00 00 00 00 00 00 NA.....
004040B0 56 58 1A 77 20 55 1A 77 UX-u U-u
004040B8 B3 D1 1C 77 F5 49 1A 77 ^Nv01-u
004040C0 77 35 1A 77 46 E0 96 77 v5-uF3w
004040C8 61 25 98 77 99 1A 77 a%wWq-u
004040D0 4A 30 97 77 53 B1 1B 77 J0wS+-w
```

The Stack window shows the following stack contents:

```
0018FF8C 771A338A #3-u RETURN to kernel32.771A338A
0018FF90 7EFDE000 .3y~
0018FF94 0018FFD4 0y1.
0018FF98 77979F72 r#wv RETURN to ntdll.77979F72
0018FF9C 7EFDE000 .3y~
0018FFA0 77E6F11E n#ew
0018FFA4 00000000 ....
0018FFA8 00000000 ....
0018FFAC 7EFDE000 .3y~
0018FFB0 00000000 ....
0018FFB4 00000000 ....
0018FFB8 00000000 ....
0018FFBC 0018FFA0 y1.
0018FFC0 00000000 ....
0018FFC4 FFFFFFFF yyyyy End of SEH chain
0018FFC8 779B71F5 0y#w SE handler
0018FFCC 0068CB9A #Eh.
0018FFD0 00000000 ....
0018FFD4 0018FFEC y1y1.
0018FFD8 77979F45 E#wv RETURN to ntdll.77979F45 from ntdll.77979F48
0018FFDC 00401050 P+Q. avtst2.<ModuleEntryPoint>
0018FFE0 7EFDE000 .3y~
0018FFE4 00000000 ....
0018FFE8 00000000 ....
0018FFEC 00000000 ....
0018FFF0 00000000 ....
0018FFF4 00401050 P+Q. avtst2.<ModuleEntryPoint>
0018FFF8 7EFDE000 .3y~
0018FFFC 00000000
```


Immunity Breakdown 2

The image shows a screenshot of the Immunity Debugger interface. The main window displays assembly code for the CPU thread of the module avtst2.exe. The code is organized into columns: address, hex, and instruction. The instructions include various assembly operations like PUSH, MOV, CALL, and JMP. On the right side of the interface, there is a menu bar with options like File, View, Debug, Plugins, ImmLib, Options, Window, Help, and Jobs. Below the menu bar, there is a toolbar with icons for various debugger functions. The bottom of the window shows the current register values, including EBP=0018FF94. Overlaid on the right side of the screenshot are several text labels with arrows pointing to specific parts of the interface: 'References' points to the assembly code; 'Hardware Breakpoints' points to the 'Hardware' tab in the 'Breakpoints' window; 'Software Breakpoints' points to the 'Software' tab in the 'Breakpoints' window; 'Call Stack' points to the 'Call Stack' window; 'Show CPU window' points to the 'CPU' window; 'Show Handles' points to the 'Handles' window; 'Window Info' points to the 'Window Info' window; 'Show Threads' points to the 'Threads' window; 'Show Memory Map' points to the 'Memory Map' window; 'Show Loaded modules' points to the 'Loaded Modules' window; and 'Show log' points to the 'Log' window.

Immunity Debugger - avtst2.exe - [CPU - main thread, module avtst2]

File View Debug Plugins ImmLib Options Window Help Jobs

Immunity: Consulting Services Manager

00401050 \$ 55 PUSH EBP
00401051 . 89E5 MOV EBP,ESP
00401053 . 6A FF PUSH -1
00401055 . 68 0C304000 PUSH avtst2.00401000
0040105A . 68 58114000 PUSH avtst2.00401150
0040105F . 64:FF35 000001 PUSH DWORD PTR FS:[0]
00401066 . 64:8925 000001 MOV DWORD PTR ES:[0],ESP
0040106D . 83EC 0C SUB ESP,0C
00401070 . 53 PUSH EBX
00401071 . 56 PUSH ESI
00401072 . 57 PUSH EDI
00401073 . 8965 E8 MOV DWORD PTR SS:[EBP-18],ESP
00401076 . 68 00000002 PUSH 20000000
0040107B . E8 900C0000 CALL avtst2.00401010
00401080 . 59 POP ECX
00401081 . A3 38434000 MOV DWORD PTR DS:[404338],EAX
00401086 . E8 E5020000 CALL avtst2.00401370
0040108B . 85C0 TEST EAX,EAX
0040108D . 75 0D JNZ SHORT avtst2.0040109C
0040108F . 6A 01 PUSH 1
00401091 . E8 1A050000 CALL avtst2.00401530
00401096 . 59 POP ECX
00401097 . E9 9B000000 JMP avtst2.00401130
0040109C > C745 FC 000001 MOV DWORD PTR SS:[EBP-4],0
004010A3 . E8 18050000 CALL avtst2.00401500
004010A8 . E8 D3050000 CALL avtst2.00401680
004010AD . E8 1E060000 CALL avtst2.004016D0
004010B2 . E8 290A0000 CALL avtst2.00401AE0
004010B7 . E8 C40A0000 CALL avtst2.00401B80
004010BC . BB 90304000 MOV EBX,avtst2.00403090
004010C1 . 81FB 90304000 CMP EBX,avtst2.00403090
004010C7 . 73 0D JNB SHORT avtst2.004010D6
EBP=0018FF94

References
Hardware Breakpoints
Software Breakpoints
Call Stack
Show CPU window
Show Handles
Window Info
Show Threads
Show Memory Map
Show Loaded modules
Show log

Software Breakpoints

Software breakpoint are CPU interrupts that can be placed anywhere in a program.

When you set a software breakpoint, the debugger simply writes the int3 instruction (opcode 0xCC) over the first byte of the target instruction. This causes an interrupt 3 to be fired whenever execution is transferred to the address you set a breakpoint on. When this happens, the debugger takes control and swaps the 0xCC opcode byte with the original first byte of the instruction so you can continue execution without hitting the same breakpoint.

If the program is not running inside a debugger, the operating system will execute default handler structured exception handler to terminate the program.

Hardware Breakpoints

Hardware breakpoints are different from their software counterparts. They use the `int1` instruction and are used for single stepping in conjunction with debug registers. There are 8 total, but only the first 4 are used. DR0-3. Sometimes it is preferable to use hardware breakpoints when normal software breakpoints fail such as when a program computes a checksum.

Software breakpoints are inserted into a program (albeit briefly) and thus, change the program. Hardware breakpoints however are not and rely on those 4 debug registers. This means however you are limited to 4 breakpoints which kind of sucks.

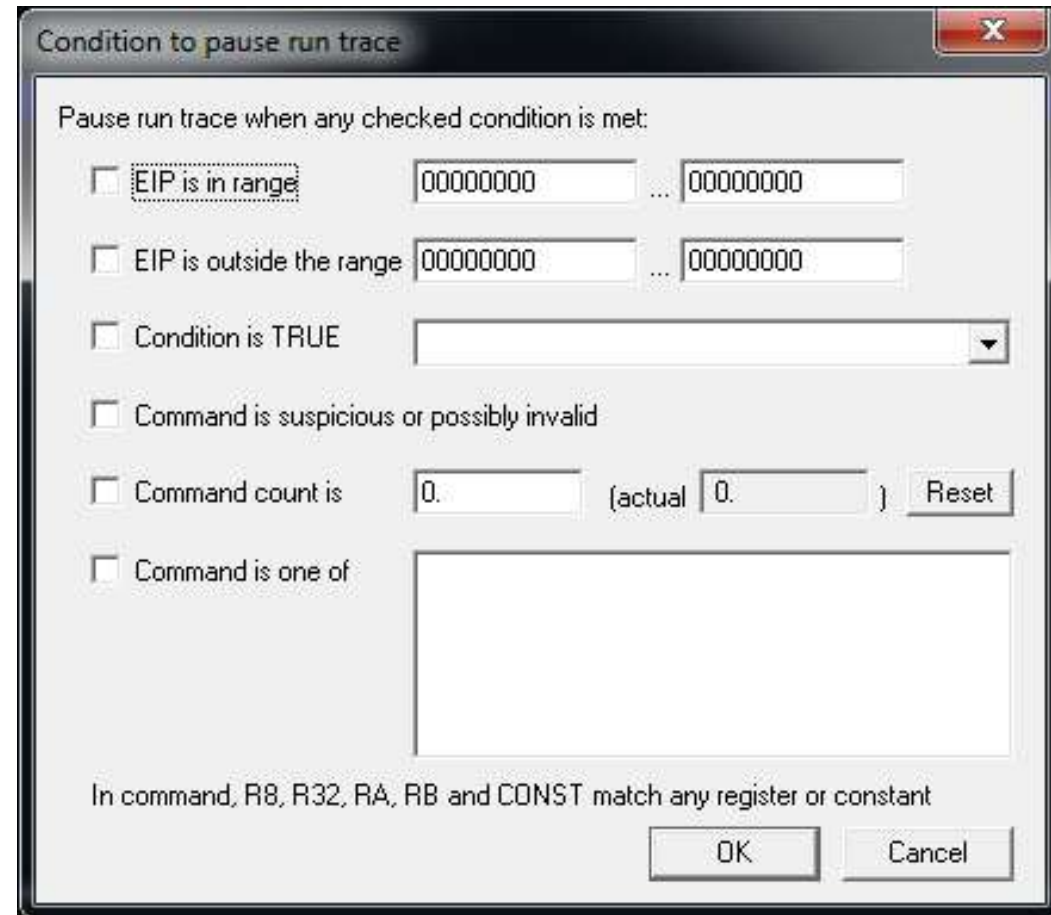
Opcodes

Opcodes are the hexadecimal representation of assembly instructions. The stuff you'll see if you were to view the executable in a hex editor. YARA rules match opcodes.

Op codes	ASM
\$ 55	PUSH EBP
. 89E5	MOV EBP,ESP
. 6A FF	PUSH -1
. 68 0C304000	PUSH avtst2.0040300C
. 68 58114000	PUSH avtst2.00401158
. 64:FF35 000000	PUSH DWORD PTR FS:[0]
. 64:8925 000000	MOV DWORD PTR FS:[0],ESP
. 83EC 0C	SUB ESP,0C
. 53	PUSH EBX
. 56	PUSH ESI
. 57	PUSH EDI
. 8965 E8	MOV DWORD PTR SS:[EBP-18],ESP
. 68 00000002	PUSH 20000000
. E8 900C0000	CALL avtst2.00401D10
. 59	POP ECX
. A3 38434000	MOV DWORD PTR DS:[404338],EAX
. E8 E5020000	CALL avtst2.00401370
. 85C0	TEST EAX,EAX
. 75 0D	JNZ SHORT avtst2.0040109C
. 6A 01	PUSH 1
. E8 1A050000	CALL avtst2.004015B0
. 59	POP ECX
. E9 9B000000	JMP avtst2.00401137
> C745 FC 000000	MOV DWORD PTR SS:[EBP-4],0
. E8 18050000	CALL avtst2.004015C0
. E8 D3050000	CALL avtst2.00401680
. E8 1E060000	CALL avtst2.004016D0
. E8 290A0000	CALL avtst2.00401AE0
. E8 C40A0000	CALL avtst2.00401B80
. BB 90304000	MOV EBX,avtst2.00403090

Tracing

A run trace is an excellent debugging technique that allows a reverse engineer to “trace” execution flow based on certain parameters. Rather than going sequentially through a program, one can define a base line, and track backwards the flow of execution by tracing where and how a program will run.



I HAVE THE POWER!!!

With a debugger you have total control over the program and you should never forget that. If you see a function that looks juicy in IDA, but see no plausible way to get there via xrefs, nothing is stopping you from jumping to it. Just modify the program's prolog to `jmp 0xwherever`. This works well especially if the function takes no arguments, otherwise except funky output or an app crash.



Jump2self

- One of those old school tricks any assembly guru will tell you where which you jump 2 bytes forward and the jump itself is 2 bytes. Jump to yourself. An infinite loop. In x86 asm, the opcodes are 0xEB 0xFE.
- From time to time I use this trick, for example when a new process is created of the same program and I want to continue execution before the app is run.

Basic Debugger Usage

Load up the program 'debuggertest1.exe' from your 'Samples' folder.



Patching

- Patching is the process of adding or removing assembly instructions in an exe without source code. It could be something as simple as changing conditional jmp to always return true/false, or it could be something complex such as hollowing out a piece of memory and injecting entire functions within and calling them.
- Patching can be done with a hex editor, a debugger, or even via the WriteProcessMemory API which is how most video game trainers do it.

Patching P2

Immunity Debugger has all of the necessary capabilities to perform on the fly patching. Simply right click and choose 'assemble'. Immunity will also auto-correct for alignment.

Saving your patches is as easy as right clicking and choosing the option 'copy to executable', then selecting the sub option 'all modifications'.

Before

```
00401000 $ 55      PUSH EBP
00401001 . 89E5      MOV EBP,ESP
00401003 > B8 64000000  MOV EAX,64
00401008 . 40          INC EAX
00401009 . BB C8000000  MOV EBX,0C8
0040100E . 43          INC EBX
0040100F . 0FAFC3      IMUL EAX,EBX
00401012 . 50          PUSH EAX
00401013 . E8 A0700000  CALL <?KERNEL32.Sleep>
00401018 . EB E9      JMP SHORT colin.00401003
0040101A . 68 00704000  PUSH colin.00407000
0040101F . E8 4C000000  CALL colin.00401070 [Arg1 = 00407000 ASCII "yeeeeaaaah"
                                colin.00401070]
00401024 . 59          POP ECX
00401025 . 31C0      XOR EAX,EAX
00401027 . 89EC      MOV ESP,EBP
00401029 . 5D          POP EBP
0040102A . C3          RETN
0040102B . 90          NOP
0040102C . 8D7C27 00  LEA EDI,DWORD PTR DS:[EDI]
00401030 . 8B4424 04  MOV EAX,DWORD PTR SS:[ESP+4]
00401034 . 50          PUSH EAX
00401035 . E8 06060000  CALL colin.00401640
0040103A . 59          POP ECX
0040103B . C3          RETN
```

After

```
00401000 $ 55      PUSH EBP
00401001 . 89E5      MOV EBP,ESP
00401003 > B8 64000000  MOV EAX,64
00401008 . 40          INC EAX
00401009 . BB C8000000  MOV EBX,0C8
0040100E . 43          INC EBX
0040100F . 0FAFC3      IMUL EAX,EBX
00401012 . 50          PUSH EAX
00401013 . 90          NOP
00401014 . 90          NOP
00401015 . 90          NOP
00401016 . 90          NOP
00401017 . 90          NOP
00401018 . 90          NOP
00401019 . 90          NOP
0040101A . 68 00704000  PUSH colin.00407000
0040101F . E8 4C000000  CALL colin.00401070 [Arg1 = 00407000 ASCII "yeeeeaaaah"
                                colin.00401070]
00401024 . 59          POP ECX
00401025 . 31C0      XOR EAX,EAX
00401027 . 89EC      MOV ESP,EBP
00401029 . 5D          POP EBP
0040102A . C3          RETN
```

Basic Malware Analysis

Here we go over a very basic sample <order_report_id_34872384972398472398.exe>

It will be in the folder named 'samples' on the drives I handed out. Open it up in your debugger / VM and we'll go over it together.

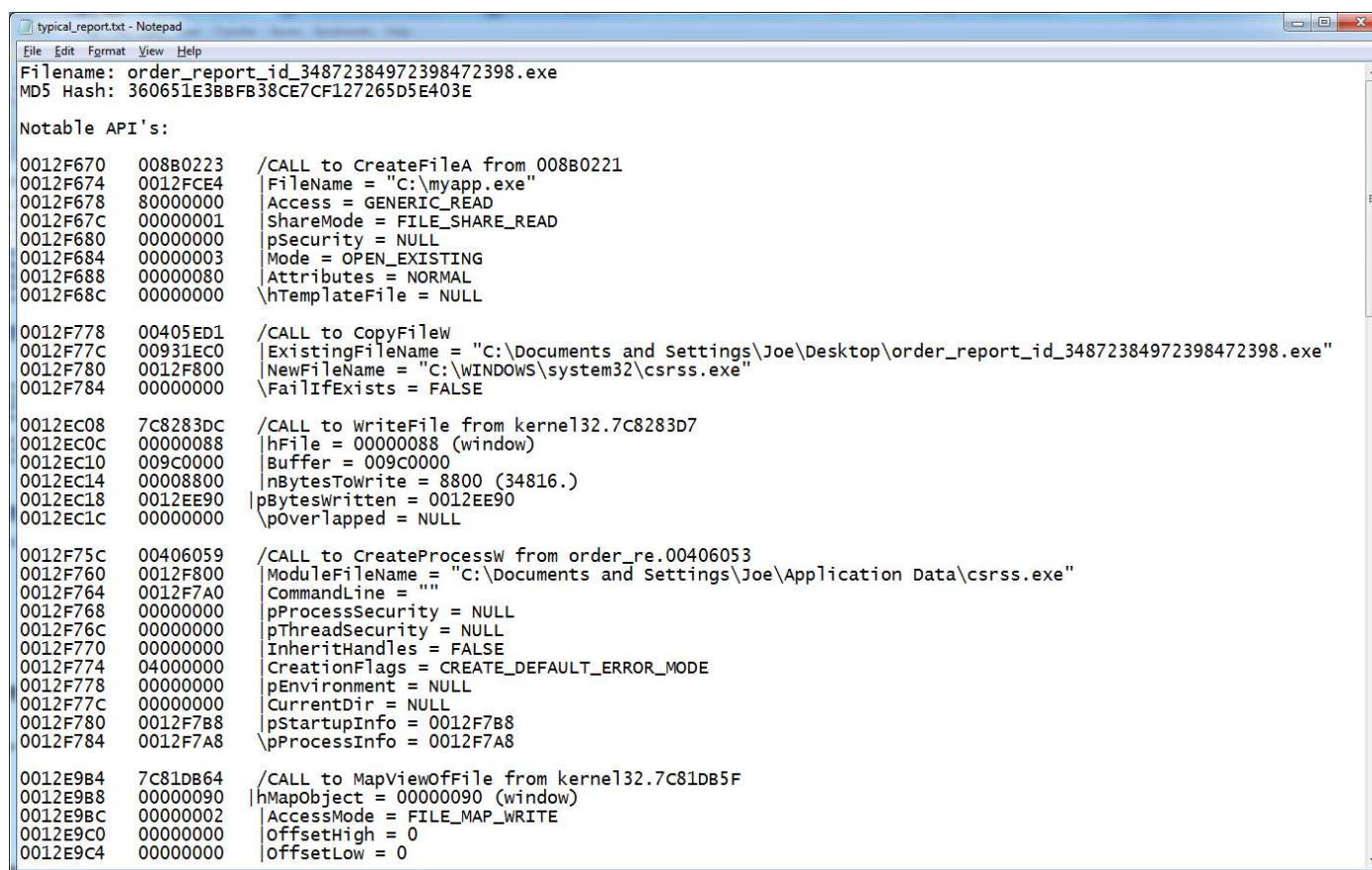


Reporting

There is no “typical” reporting structure when it comes to classifying / documenting malware, however I like to keep my reports with the most essential fields such as the MD5, API’s, C&C IP’s, and IOC’s

Its good to keep reports descriptive, but do not go overboard.

Example of one of my reports: http://gironsec.com/typical_report.txt



```
typical_report.txt - Notepad
File Edit Format View Help
Filename: order_report_id_34872384972398472398.exe
MD5 Hash: 360651E3BBFB38CE7CF127265D5E403E

Notable API's:

0012F670 008B0223 /CALL to CreateFileA from 008B0221
0012F674 0012FCE4 |FileName = "C:\myapp.exe"
0012F678 80000000 |Access = GENERIC_READ
0012F67C 00000001 |ShareMode = FILE_SHARE_READ
0012F680 00000000 |pSecurity = NULL
0012F684 00000003 |Mode = OPEN_EXISTING
0012F688 00000080 |Attributes = NORMAL
0012F68C 00000000 |hTemplateFile = NULL

0012F778 00405ED1 /CALL to CopyFileW
0012F77C 00931EC0 |ExistingFileName = "C:\Documents and Settings\Joe\Desktop\order_report_id_34872384972398472398.exe"
0012F780 0012F800 |NewFileName = "C:\WINDOWS\system32\csrss.exe"
0012F784 00000000 |FailIfExists = FALSE

0012EC08 7C8283DC /CALL to WriteFile from kernel32.7C8283D7
0012EC0C 00000088 |hFile = 00000088 (window)
0012EC10 009C0000 |Buffer = 009C0000
0012EC14 00008800 |nBytesToWrite = 8800 (34816.)
0012EC18 0012EE90 |pBytesWritten = 0012EE90
0012EC1C 00000000 |pOverlapped = NULL

0012F75C 00406059 /CALL to CreateProcessW from order_re.00406053
0012F760 0012F800 |ModuleFileName = "C:\Documents and Settings\Joe\Application Data\csrss.exe"
0012F764 0012F7A0 |CommandLine = ""
0012F768 00000000 |pProcessSecurity = NULL
0012F76C 00000000 |pThreadSecurity = NULL
0012F770 00000000 |InheritHandles = FALSE
0012F774 04000000 |CreationFlags = CREATE_DEFAULT_ERROR_MODE
0012F778 00000000 |pEnvironment = NULL
0012F77C 00000000 |CurrentDir = NULL
0012F780 0012F7B8 |pStartupInfo = 0012F7B8
0012F784 0012F7A8 |pProcessInfo = 0012F7A8

0012E9B4 7C81DB64 /CALL to MapViewOfFile from kernel32.7C81DB5F
0012E9B8 00000090 |hMapObject = 00000090 (window)
0012E9BC 00000002 |AccessMode = FILE_MAP_WRITE
0012E9C0 00000000 |OffsetHigh = 0
0012E9C4 00000000 |OffsetLow = 0
```

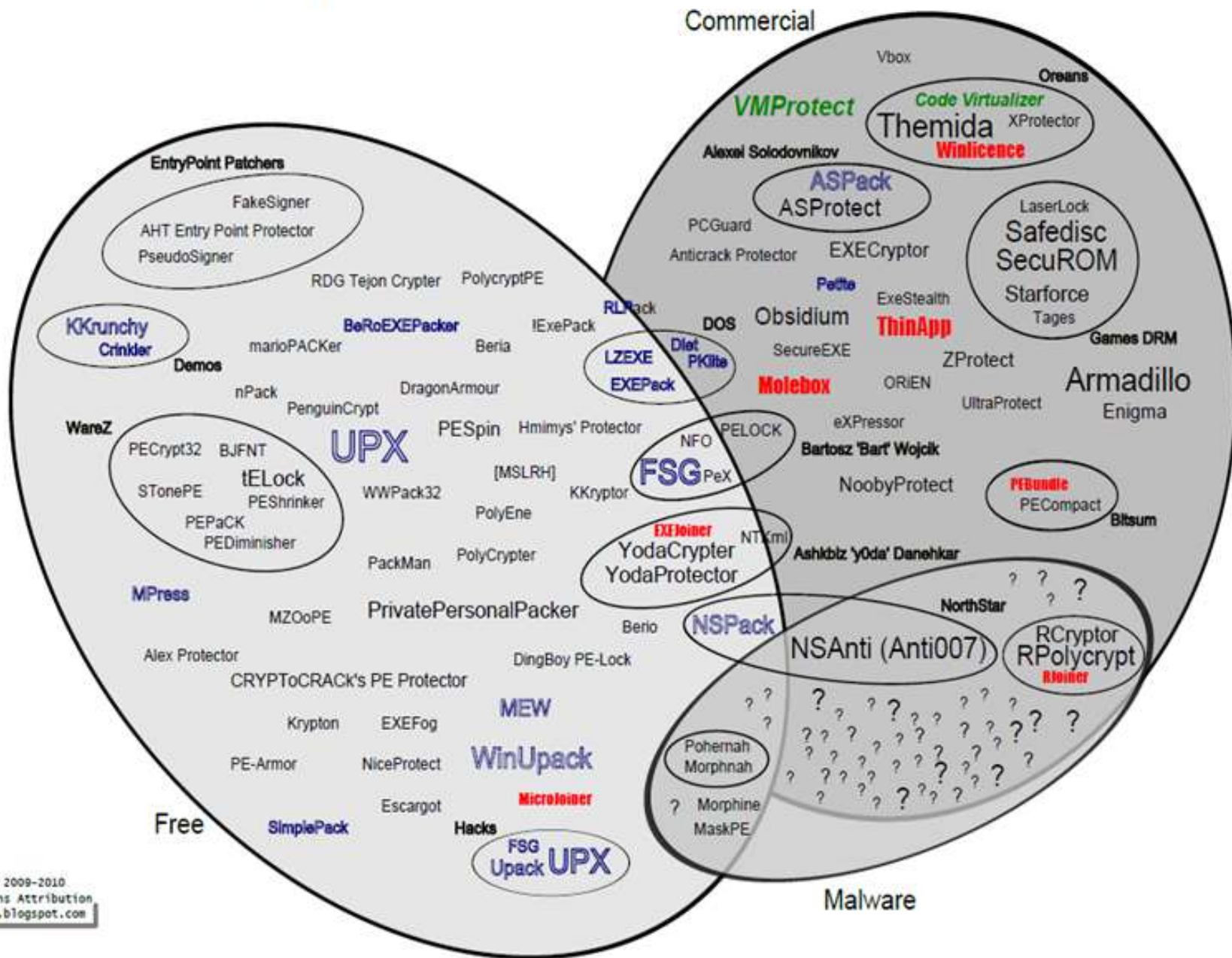
Unpacking 101

- Most malware today is packed in some way to help get around AV signature detection
- There are over 8000 known packers out there, each with their own signatures.
- They can range from simple compression to full on encryption / debugger detection and generally make the life of the Malware Reverser a pain.
- Packers are not fool proof – the exe HAS to be decrypted / decompressed at SOME point in order to run on the OS.

Packers Landscape

Bundlers
Virtualisers
Compressors

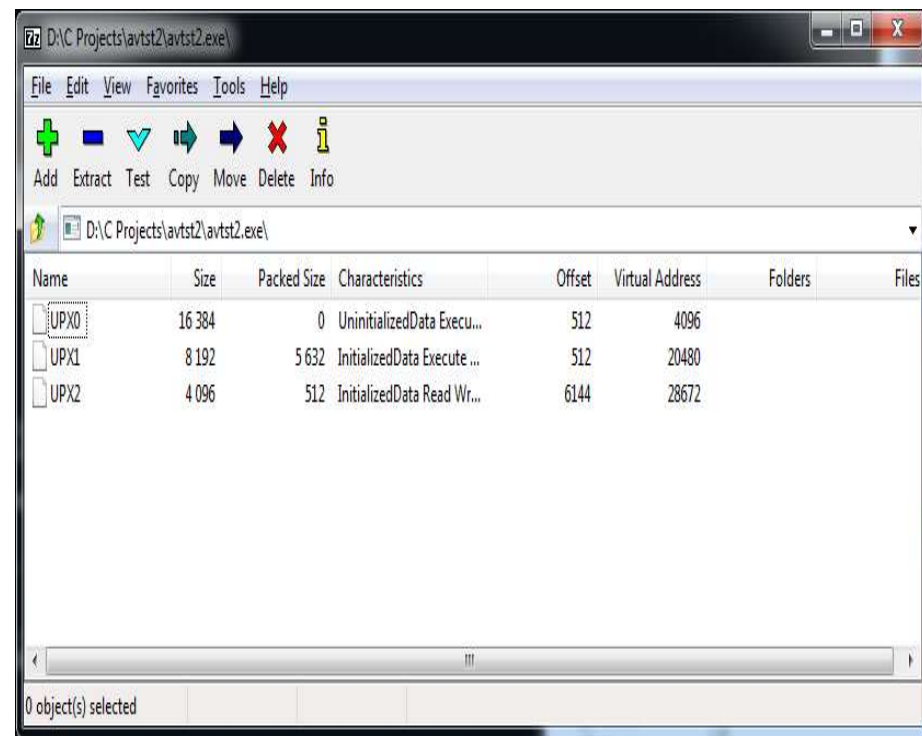
So many packers...



Unpacking p2

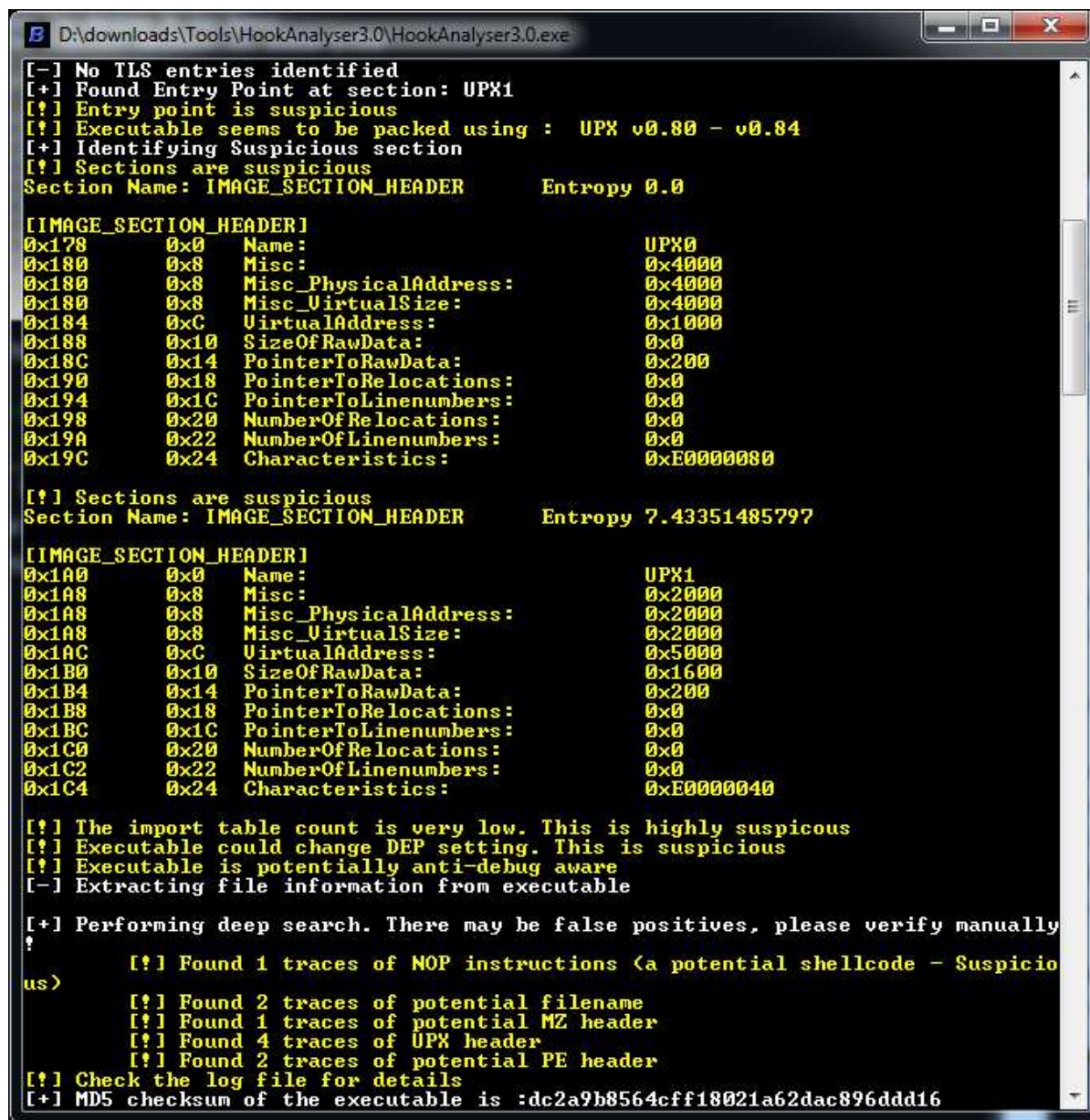
Before we start to unpack, let's go over how we identify a program is packed to begin with.

- If in IDA we see a bunch of nothing such as hardly any imports, lots of BS in the data section.
- We see odd named section headers. One can use 7zip sometimes to view section headers of executables, but generally you can do this with CFF explorer or IDA.
- HookAnalyser comes with a decent packer database with something like 4400 different packer signatures.
- Hands down the best packer identifier in my opinion is exeinfo. it beats HookAnalyser's db with a grand total of 7075 packer signatures. Not only will it tell you who packed an exe, it will provide a link for how to unpack it. Thanks guys!
- Once we know what kind of packer is used (if any) we can begin our unpacking. Be forewarned through, some packers are hard to get around. Really hard. At some point it becomes not worth it in which case we might want to skip the entire reversing process from a debugger level and instead opt for the live analysis tools to save time and a major migraine. Don't feel bad if you can't get around a packer, it happens to me from time to time.



Unpacking p3

Pic on the left demonstrates HookAnalyzer running an analysis on a UPX packed binary. It's not a perfect tool, if you'll notice the false positive of supposed NOP instructions. The nice thing about HookAnalyser is it will extract IP addresses for you automatically – very useful for malware analysis.



```
D:\downloads\Tools\HookAnalyser3.0\HookAnalyser3.0.exe
[-] No ILS entries identified
[+] Found Entry Point at section: UPX1
[!] Entry point is suspicious
[!] Executable seems to be packed using : UPX v0.80 - v0.84
[+] Identifying Suspicious section
[!] Sections are suspicious
Section Name: IMAGE_SECTION_HEADER      Entropy 0.0

[IMAGE_SECTION_HEADER]
0x178      0x0      Name:                                UPX0
0x180      0x8      Misc:                                0x4000
0x180      0x8      Misc_PhysicalAddress:                0x4000
0x180      0x8      Misc_VirtualSize:                    0x4000
0x184      0xC      VirtualAddress:                      0x1000
0x188      0x10     SizeOfRawData:                       0x0
0x18C      0x14     PointerToRawData:                    0x200
0x190      0x18     PointerToRelocations:                0x0
0x194      0x1C     PointerToLinenumbers:                0x0
0x198      0x20     NumberOfRelocations:                0x0
0x19A      0x22     NumberOfLinenumbers:                0x0
0x19C      0x24     Characteristics:                    0xE0000080

[!] Sections are suspicious
Section Name: IMAGE_SECTION_HEADER      Entropy 7.43351485797

[IMAGE_SECTION_HEADER]
0x1A0      0x0      Name:                                UPX1
0x1A8      0x8      Misc:                                0x2000
0x1A8      0x8      Misc_PhysicalAddress:                0x2000
0x1A8      0x8      Misc_VirtualSize:                    0x2000
0x1AC      0xC      VirtualAddress:                      0x5000
0x1B0      0x10     SizeOfRawData:                       0x1600
0x1B4      0x14     PointerToRawData:                    0x200
0x1B8      0x18     PointerToRelocations:                0x0
0x1BC      0x1C     PointerToLinenumbers:                0x0
0x1C0      0x20     NumberOfRelocations:                0x0
0x1C2      0x22     NumberOfLinenumbers:                0x0
0x1C4      0x24     Characteristics:                    0xE0000040

[!] The import table count is very low. This is highly suspicious
[!] Executable could change DEP setting. This is suspicious
[!] Executable is potentially anti-debug aware
[-] Extracting file information from executable

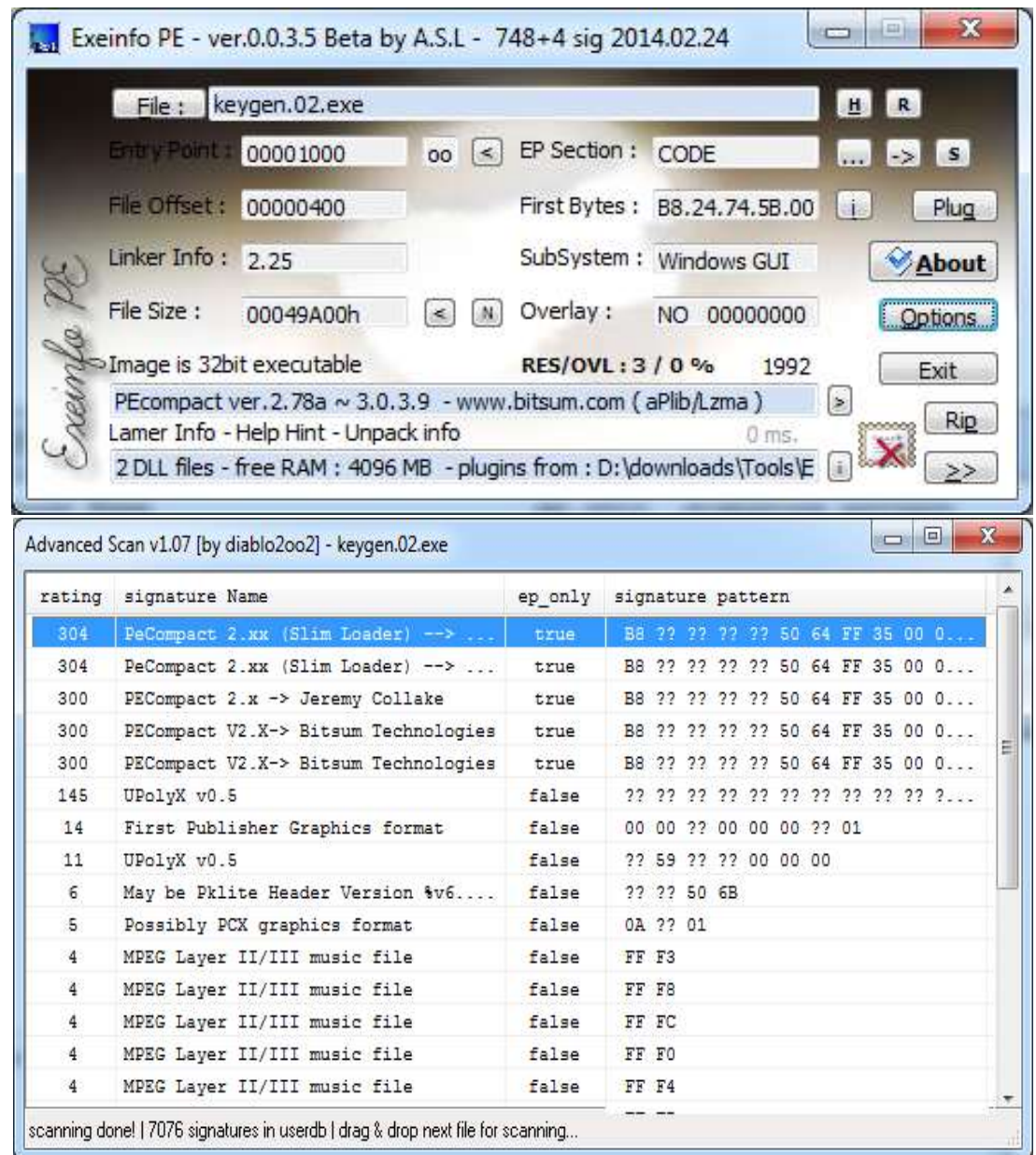
[+] Performing deep search. There may be false positives, please verify manually
!
      [!] Found 1 traces of NOP instructions (a potential shellcode - Suspicious)
      [!] Found 2 traces of potential filename
      [!] Found 1 traces of potential MZ header
      [!] Found 4 traces of UPX header
      [!] Found 2 traces of potential PE header
[!] Check the log file for details
[+] MD5 checksum of the executable is :dc2a9b8564cff18021a62dac896ddd16
```


Unpacking p4

When it comes to identifying packers, there is no tool better than exeinfo, second only to experience.

It has a packer database with which it searches through, can scan for media files, IP addresses, hostnames, and all that other good stuff.

And its free which makes it #1.



Unpacking demonstration

Fire up your debuggers. We'll be going over the 'Local-Appwizard.exe' file in the 'Samples' folder.



Dynamic Analysis

- For when you either a) don't care about the inner workings and only want basic info from the malware or b) malware packed /encrypted tight and you cant get around it.
- This is where proper usage of procmon / wireshark / cuckoo are a godsend. Even if a program's assembly code isn't available, we can still see what its doing and probe it blackbox style.

Memory Analysis

- HookAnalyzer and Volatility are excellent memory analysis tools. HookAnalyzer will do it live, but Volatility requires a memory dump and has to be done post execution.
- Process Hacker allows you to dump a running process's memory for inspection and allows for filtering and searching. The same can be done via your debugger, but this isn't always feasible.

Hook Analyser

This tool supports live memory analysis of running processes This makes it indispensable.

```
C:\Documents and Settings\Joe\Desktop\HookAnalyser3.0\HookAnalyser3.0.exe
[-] Enter the process id : 3892
[!] Displaying Modules for the process - 3892

Address      Size      Path
00400000     0000b000 C:\Documents and Settings\Joe\Desktop\order_report_id_34
872384972398472398.exe
7c900000     000af000 C:\WINDOWS\system32\ntdll.dll
7c800000     000f6000 C:\WINDOWS\system32\kernel32.dll
77dd0000     0009b000 C:\WINDOWS\system32\ADVAPI32.DLL
77e70000     00092000 C:\WINDOWS\system32\RPCRT4.dll
77fe0000     00011000 C:\WINDOWS\system32\Secur32.dll
5ad60000     0000a000 C:\WINDOWS\system32\UDDBG.DLL
7e410000     00091000 C:\WINDOWS\system32\USER32.dll
77f10000     00049000 C:\WINDOWS\system32\GDI32.dll
77f60000     00076000 C:\WINDOWS\system32\SHLWAPI.dll
77c10000     00058000 C:\WINDOWS\system32\msvcrt.dll
71ab0000     00017000 C:\WINDOWS\system32\WS2_32.dll
71aa0000     00008000 C:\WINDOWS\system32\WS2HELP.dll
7c9c0000     00017000 C:\WINDOWS\system32\SHELL32.dll
773d0000     00103000 C:\WINDOWS\WinSxS\x86_Microsoft.Windows.Common-Controls_
6595b64144ccf1df_6.0.2600.5512_x-ww_35d4ce83_comctl32.dll
5d090000     0009a000 C:\WINDOWS\system32\comctl32.dll
774e0000     0013d000 C:\WINDOWS\system32\ole32.dll
77b40000     00022000 C:\WINDOWS\system32\Apphelp.dll
77c00000     00008000 C:\WINDOWS\system32\VERSION.dll

2014-10-20-04-37-37.txt - Notepad
File Edit Format View Help
[+] Searching for TLS entries..
[-] No TLS entries identified
[+] Found Entry Point at section: .text
[!] Suspicious

[!] Executable seems to be packed using unknown packer[+] Identifying Suspicious
section
[!] Following sections entropies are suspicious

[!] The import table count is very low. This is highly suspicious[!] Executable is
potentially anti-debug aware
[!] Found 4 traces of potential filename
[-] At the offset 00007734, found: '.DLL'
7710 63 41 64 64 72 65 73 73 00 00 00 00 55 70 64 61 cAddress....Upda
7720 74 65 52 65 73 6F 75 72 63 65 57 00 4B 45 52 4E teResourcew.KERN
7730 45 4C 33 32 2E 44 4C 4C 00 00 00 00 4C 6F 6F 6B EL32.DLL....Look
7740 75 70 53 65 63 75 72 69 74 79 44 65 73 63 72 69 upSecurityDescri
7750 70 74 6F 72 50 61 72 74 73 41 00 00 00 00 4C 73 ptorPartsA....Ls

[-] At the offset 00008272, found: '.DLL'
8250 79 44 65 73 63 72 69 70 74 6F 72 54 6F 41 63 63 yDescriptorToAcc
8260 65 73 73 4E 61 6D 65 64 41 00 41 44 56 41 50 49 essNamedA.ADVAPI
8270 33 32 2E 44 4C 4C 00 00 00 00 56 44 4D 45 6E 75 32.DLL....VDMENU
8280 6D 50 72 6F 63 65 73 73 57 4F 57 00 00 00 56 44 mProcessWOW...VD
8290 4D 47 65 74 53 65 67 6D 65 6E 74 49 6E 66 6F 00 MGetSegmentInfo.

[-] At the offset 000083B8, found: '.DLL'
8390 00 00 56 44 4D 47 65 74 53 79 6D 62 6F 6C 00 00 ..VDMGetSymbol..
83A0 00 00 56 44 4D 45 6E 75 6D 54 61 73 6B 57 4F 57 ..VDMEnumTaskWOW
83B0 00 00 56 44 4D 44 42 47 2E 44 4C 4C 00 00 00 00 ..UDDBG.DLL....
83C0 44 69 61 6C 6F 67 42 6F 78 50 61 72 61 6D 57 00 DialogBoxParamw.
83D0 00 00 53 65 6E 64 4D 65 73 73 61 67 65 41 00 00 ..SendMessageA..

[-] At the offset 000083E6, found: '.DLL'
83C0 44 69 61 6C 6F 67 42 6F 78 50 61 72 61 6D 57 00 DialogBoxParamw.
83D0 00 00 53 65 6E 64 4D 65 73 73 61 67 65 41 00 00 ..SendMessageA..
83E0 55 53 45 52 33 32 2E 44 4C 4C 00 00 00 00 00 00 USER32.DLL.....
83F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
8400 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

[!] Found 1 traces of potential MZ header
[-] At the offset 00000000, found: 'MZ'
0000 4D 5A 90 00 03 00 00 04 00 00 00 FF FF 00 00 MZ.....
0010 B8 00 00 00 00 00 00 00 40 00 00 00 00 00 00 00 .....@.....
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

[!] Found 1 traces of potential PE header
[-] At the offset 000000E0, found: 'PE'
00C0 3D 03 13 8B C6 20 2E 8B 50 03 6B 8B C6 20 2E 8B =....P.k..
00D0 52 69 63 68 C7 20 2E 8B 00 00 00 00 00 00 00 00 Rrch. ....
```


Process Hacker

Another must have useful tool. Capable of live memory searching and dumping, it provides things taskmanager should but doesn't including disk IO monitoring and networking activity monitoring capabilities.

The screenshot displays the Process Hacker application with two windows open. The main window, titled 'order_report_id_34872384972398472398.exe (3332) Properties', has the 'Strings' tab selected. It shows a list of memory strings with columns for Name, Address, Size, and Protection. The strings include various system paths and application data, such as 'Free', 'Private (Commit)', 'Mapped (Commit)', and 'unicode.nls: Mapped (Co...)'. A 'Refresh' button is visible at the top right of the strings list.

The second window, titled 'Results - order_report_id_34872384972398472398.exe (3332)', displays the results of a memory search. It shows 313 results, with columns for Address, Length, and Result. The results include system paths like 'ALLUSERSPROFILE=C:\Documents a...', 'APPDATA=C:\Documents and Settin...', and 'CLIENTNAME=Console'. A third window, titled 'Results - order_report_id_34872384972398472398.exe (3332)', displays 45 results, with columns for Address, Length, and Result. The results include system paths like 'ComSpec=C:\WINDOWS\system32\...', 'C:\Documents and Settings\Joe\Des...', and 'C:\Documents and Settings\Joe\De...'. The results window also has a 'Filter' button and a 'Save...' button.

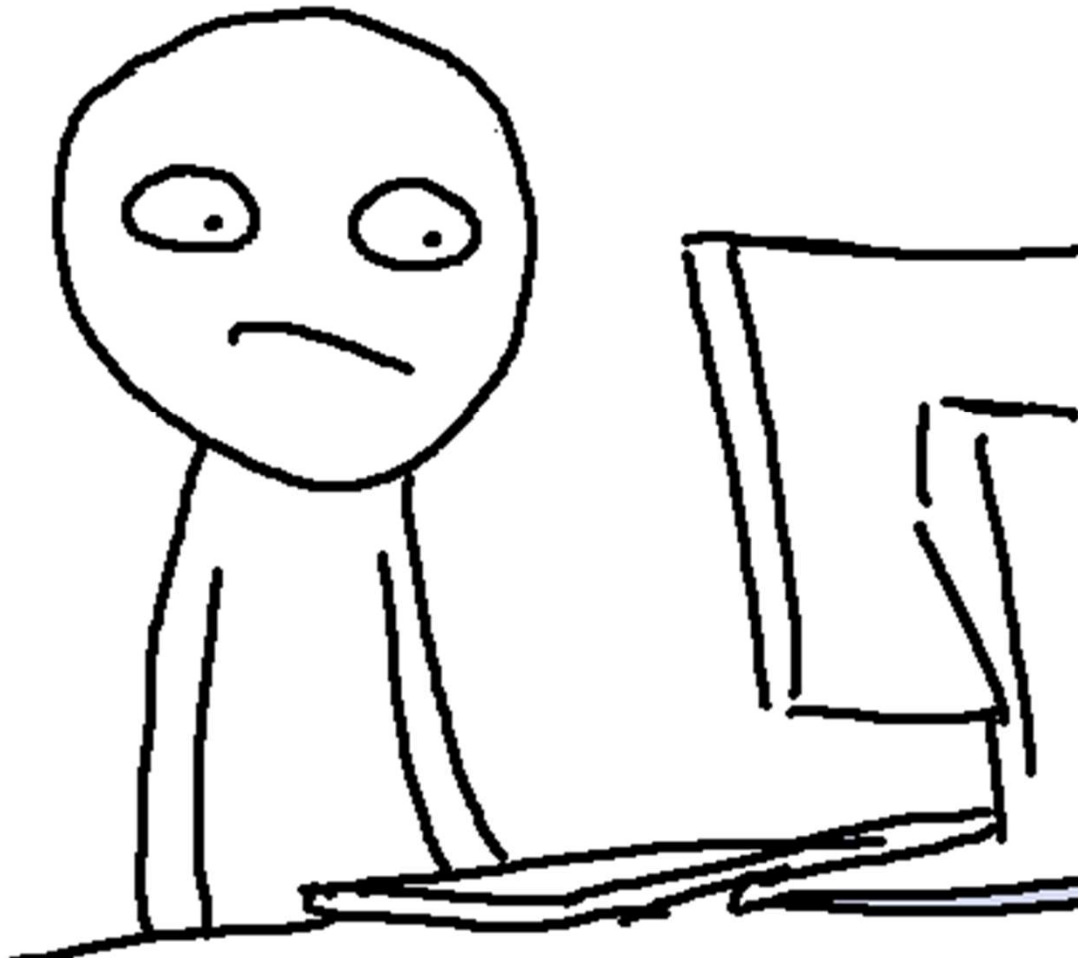
Name	Address	Size	Protect...
Free	0x0	64 kB	NA
Private (Commit)	0x10000	4 kB	RW
Free	0x11000	60 kB	NA
Private (Commit)	0x20000	4 kB	RW
Free	0x21000	60 kB	NA
Private (Reserve)	0x30000	1,004 kB	
Private (Commit)	0x12b000	4 kB	RW+G
Private (Commit)	0x12c000	16 kB	RW
Mapped (Commit)	0x130000	12 kB	R
Free	0x133000	52 kB	NA
Private (Commit)	0x140000	72 kB	RW
Private (Reserve)	0x152000	952 kB	
Private (Commit)	0x240000	24 kB	RW
Private (Reserve)	0x246000	40 kB	
Mapped (Commit)	0x250000	12 kB	RW
Mapped (Reserve)	0x253000	52 kB	
unicode.nls: Mapped (Co...)	0x260000	88 kB	R
Free	0x276000	40 kB	NA
locale.nls: Mapped (Com...)	0x280000	260 kB	R
Free	0x2c1000	60 kB	NA
sortkey.nls: Mapped (Co...)	0x2d0000	260 kB	R
Free	0x311000	60 kB	NA
sorttbls.nls: Mapped (Co...)	0x320000	24 kB	R
Free	0x326000	40 kB	NA
Mapped (Commit)	0x330000	20 kB	RX
Mapped (Reserve)	0x335000	748 kB	
Mapped (Commit)	0x3f0000	8 kB	RX
Mapped (Reserve)	0x3f2000	24 kB	

Address	Length	Result
0x10010	102	ALLUSERSPROFILE=C:\Documents a...
0x10078	108	APPDATA=C:\Documents and Settin...
0x100e6	36	CLIENTNAME=Console
0x1010c	96	CommonProgramFiles=C:\Program Fi...
0x1016e	56	COMPUTERNAME=JOE-58BAEB57682
0x101a8	70	ComSpec=C:\WINDOWS\system32\...
0x101f0	38	FP_NO_HOST_CHECK=NO
0x10218	24	HOMEDRIVE=C:

Address	Length	Result
0x101a8	70	ComSpec=C:\WINDOWS\system32\...
0x205bc	156	C:\Documents and Settings\Joe\Des...
0x2065c	162	"C:\Documents and Settings\Joe\De...
0x20700	156	C:\Documents and Settings\Joe\Des...
0x12d674	100	ents and Settings\Joe\Application D...
0x12d904	76	tings\Joe\Application Data\svchost...
0x12dc0a	22	svchost.exe
0x12dec0	22	svchost.exe
0x12e128	116	C:\Documents and Settings\Joe\AppData...
0x12ea90	116	C:\Documents and Settings\Joe\AppData...
0x12f0c0	28	2398472398.exe
0x12f57c	202	C:\Documents and Settings\Joe\AppData...
0x12f800	116	C:\Documents and Settings\Joe\AppData...
0x141e94	152	\Documents and Settings\Joe\Deskt...
0x1420a4	120	?C:\Documents and Settings\Joe\A...
0x14211e	38	384972398472398.exe
0x1423c8	81	"C:\Documents and Settings\Joe\De...
0x1448e8	78	C:\Documents and Settings\Joe\Des...

Volatility In Action

- <video of me using volatility or I'll do it live>



Where Do I Get Malware?

- <http://www.offensivecomputing.net/>
- IRC
- Reddit / 4chan / Tumblr
- me (joe@gironsec.com),
- Twitter #malwaremustdie
- AV companies
- <http://syrianmalware.net>
- Torrents / cracked software / gnutella network.
- Spam email

Help! I'm stuck ☹️

If you ever get stuck on a sample, fear not. There are communities out there that deal with malware forensics. Reddit, Exeforums, woodman, kernelmode.info, mailing lists, and even twitter are excellent sources full of people who will help. Even IRC still works.



Additional Resources

- <https://code.google.com/p/corkami/>
- ^^ An excellent resource for info on reversing
- [http://www.woodmann.com/collaborative/tools/index.php/Category:RCE Tools](http://www.woodmann.com/collaborative/tools/index.php/Category:RCE_Tools)
- ^^ huge resource for reverse engineering tools
- <http://reddit.com/r/ReverseEngineering/>
- ^^ Still better than /r/malware. Avoid /r/malware
- <http://gironsec.com/blog/>
- ^^ shameless self promotion? No, I got plenty of good guides!

Questions?



Contact Info

Name: Joe Giron

Email: Joe@gironsec.com

Phone: 512-902-3787

